Know2Act: When Does an Agent's Written Prediction Control Its Next Action?

Yiwen Lu
University of Notre Dame
ylu37@nd.edu
Manuscript, 2026

Code, protocol texts and per-model read-outs: https://github.com/EvenEureka/Know2Act

Abstract

Agents are often trained to write a prediction before acting, which people read to monitor or steer the agents. We ask whether such a prediction controls the next action when the knowledge it states is also in the weights, and how the fine-tuning data decide this. In a counterfactual ALFWorld whose object locations only fine-tuning can teach, Qwen3-4B agents write a one-line location prediction before each search. When training predictions are always followed, deleting the prediction at the first search step lowers the predicted move's probability by 0.015. Replacing it with the agent's prediction for another object redirects 80% of first steps. A randomly ordered ranking, never seen in training, redirects even more (90%), and, in held-out rooms, objects whose location the weights cannot know are redirected just as often. When a quarter or half of the training predictions are contradicted by the action, redirection falls to 1.2% and 0.3%, as a perfect imitator of those data would. In ALF-CF, when the training data leave open whether the action follows the prediction or the learned convention, fine-tuning produces agents that follow the prediction; contradicting a quarter of the training predictions removes this preference while the agents keep writing predictions.

1 Introduction

Language-model agents are often trained to say what they expect before they act. ReAct-style thoughts, belief states and plans all put a prediction in writing (Yao et al., 2023; Wang et al., 2026; Zhang et al., 2025). Monitors read these predictions to judge why an agent acts (Korbak et al., 2025), and editing them is a cheap way to steer an agent. Both practices assume the written prediction takes part in the decision.

Work on chain-of-thought faithfulness shows that whether this assumption holds depends on the test. Deleting reasoning often leaves answers unchanged, while editing it can change them (Lanham et al., 2023). In fine-tuned agents, replacing the whole reasoning trace moves fewer actions as training goes on (Liu et al., 2026).

For knowledge that the model has learned, the question is harder. In ordinary training data the written prediction and the knowledge in the weights rarely conflict, so the data do not show which of the two the action follows when they do.

We study the question in a setting built so that the two can be told apart. ALF-CF is a counterfactual version of ALFWorld in which each object class is kept in a fixed, arbitrary receptacle class, chosen so that neither the original data nor the base model's prior points to it. The location can therefore only be learned from fine-tuning.

At the first search step of an episode, the agent has not yet seen the target, so its one-line prediction, which we call the *note*, is the only location information in its context (Figure 1). We fine-tune Qwen3-4B on six versions of the training data. They differ in one respect each: whether predictions are always followed by the action, sometimes contradicted by it, or followed by an action that ignores them.

At the first search step we then either delete the agent's prediction or replace it with the prediction the agent writes for another object, and record where the agent goes. Each arm is compared with what a perfect imitator of its training data would do, so effects that follow from the data construction are not read as properties of the model.

When every prediction in the training data is followed, the prediction steers the agent without being needed. Deleting the prediction lowers the probability of the predicted move by 0.015. Replacing it redirects 80% of first moves across eight seeds, and a randomly ordered ranking, a form never seen in training, is followed even more readily (0.90). In held-out rooms, whether the weights hold a convention for the object makes no detectable difference.

Over a whole episode, writing another object's prediction at every step raises the number of re-

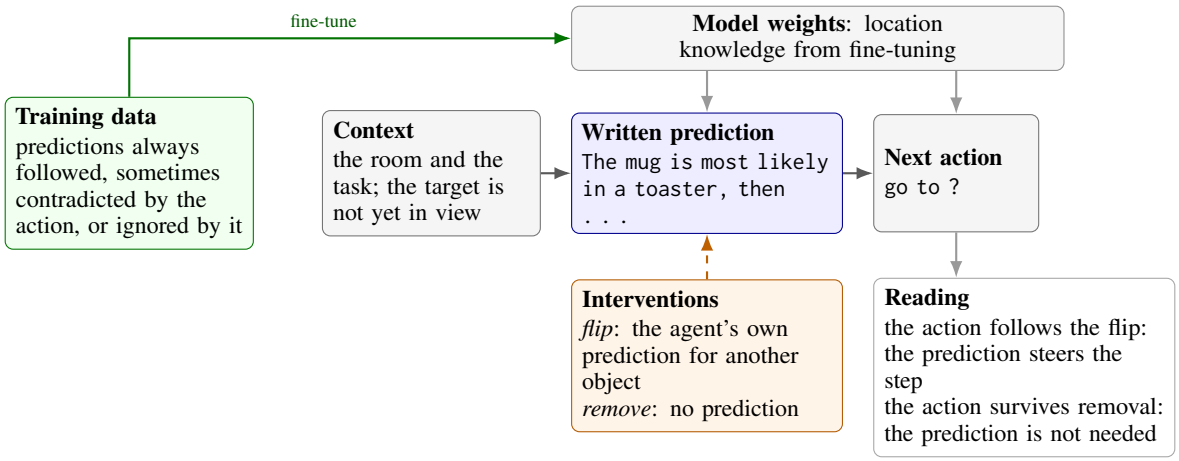


Figure 1: The question and the design. Before each search step the agent writes where it expects the target. Fine-tuning puts the location knowledge into the weights, from which it can reach the action directly or through the written prediction. We fine-tune on data that differ only in how predictions and actions are coupled. At the first search step, where the prediction is the only location information in context, we flip or remove the prediction and record where the agent goes. Each training arm is also compared with what a perfect imitator of its data would do.

ceptacles visited from about two to about nine. The control is strongest at the first step and much weaker later, where it depends on what the search has already shown. Removing the agent's earlier predictions from its history partly restores the control.

When a quarter or half of the training predictions are contradicted by the teacher's action, the agents keep writing predictions but follow a replaced one in at most about 1% of first steps, within 0.012 of what a perfect imitator of such data does. When half of the actions ignore correct predictions instead, redirection drops to 27%, between what an imitator would show if it credited its correct actions to the weights (0) and to the predictions (0.50).

For monitoring, this means that when the training data also teach the agent to act without a prediction, as ours do at 30% of steps, a deletion test calls the predictions decorative, while an edit test shows that they steer the first decision. For training, contradicting a quarter of the predictions was enough to remove that steering.

2 Related Work

Intervening on written reasoning. The standard way to ask whether a model uses its chain of thought is to change the text and watch the answer. Lanham et al. (2023) truncate the reasoning, insert mistakes, replace it with filler tokens and paraphrase it. How much the answer depends on the reasoning varies widely across tasks. Turpin et al. (2023) show that explanations can omit the feature that actually drove the answer. Our three main conditions descend from these probes. Removing the note corresponds to early answering, replacing it with another class's prediction corresponds to inserted mistakes, and the content-free note corresponds to filler. We apply these probes to a different object: a single line in which an agent states where it expects an object, at a point where nothing else in its context bears on that question.

Reasoning in fine-tuned agents. The closest work is Liu et al. (2026). They fine-tune Qwen3 agents on ALFWorld, ScienceWorld and BFCL trajectories and replace an agent's reasoning with a trace that supports a different action. With more training, their agents more often keep the prompt-based action under the substituted trace, whereas our agents trained on always-followed predictions (N100, Section 4) follow a substituted note at 80% of first steps. Their traces are free-form reasoning, while our predictions are one-line rankings whose first item names the receptacle type of the next action, so an agent can follow our predictions by copying. Our design cannot tell which difference matters. Liu et al. (2026) also propose a training lever that masks the loss on the final action tokens for 30% of samples, under SFT and GRPO. They report that it improves out-of-domain performance, most in ALFWorld and ScienceWorld. Our N100M arm targets the same shortcut with a different mask: it removes the action loss where no note is written, rather than on a random 30% of samples.

Three other studies edit an agent's reasoning and read off its actions. Sediqin and Fatemi (2026) replace the private reasoning of frontier models at saved checkpoints and find that a replaced statement of intent changes the next action most of the time. Kim et al. (2025) edit a thought in a single ALFWorld episode. Wang et al. (2026) append the true environment state to an embodied agent's history without editing its stated beliefs.

None of these studies controls what the written text could know, uses as the edit the agent's own prediction for another object in the same context, or compares the result with what a perfect imitator of the training data would do. Our design adds all three.

Training reasoning to be used. FRODO (Paul et al., 2024) trains small models so that answers depend on their rationales, using a causal reward and counterfactual preference data. CSR (Akter et al., 2025) and FRIT (Swaroop et al., 2025) train sensitivity to perturbed reasoning on question answering and math. Agent work has used predictions mainly as training objectives. Early Experience (Zhang et al., 2025) predicts next states during training but not at inference. Wang (2026) find that the content of auxiliary prediction targets matters less than the channel through which they are trained. We hold the training objective fixed and vary only how the data couple the note to the action.

Which cue does fine-tuning adopt when the data do not decide? When two cues predict the training labels equally well, fine-tuned models adopt one of them, often the more easily extracted, and a small share of disambiguating examples can switch them (Warstadt et al., 2020; Lovering et al., 2021). This is one source of shortcut learning (Geirhos et al., 2020). N100 is such a case for an agent: the note and the learned convention always agree, and the agent adopts the note. The arms with contradicted notes add disambiguating examples.

Context versus parametric knowledge. A flipped note pits text in the context against what the weights have learned. Question-answering models over-rely on parametric knowledge when a passage contradicts it (Longpre et al., 2021). Large models, in contrast, accept coherent counter-evidence and fall back on memory when the evidence is mixed (Xie et al., 2024). Our setting differs in two ways. The conflicting text is the agent's own kind of output, and for one group of object classes the weights hold nothing to resist with.

Monitoring and steering through written reasoning. Whether written reasoning is load-bearing matters for monitoring. Emmons et al. (2025) separate reasoning as rationalization from reasoning as computation. Korbak et al. (2025) argue that monitorability depends on training choices. In our N100 agents, deleting the note barely changes the action while editing it redirects the agent, and the training data decide whether this happens.

Counterfactual environments. Changing the rules of a familiar task to separate learned knowledge from priors has precedents in planning (Valmeekam et al., 2023) and in counterfactual task variants (Wu et al., 2024). ALF-CF applies the same idea to object placement in ALFWorld (Shridhar et al., 2021).

3 A World Where Location Knowledge Comes Only From Fine-Tuning

Agent and notes. We use the text version of ALFWorld (Shridhar et al., 2021), built on TextWorld (Côté et al., 2018) from ALFRED scenes (Shridhar et al., 2020). The opening observation lists the receptacles in the room and states the task, for example *put a clean mug in the coffeemachine*. An object becomes visible only after the agent goes to the receptacle that holds it, and opens that receptacle if it is closed. Until the target has been seen, the context therefore carries no information about where it is.

We call every `go to` that the agent takes in this state a *fallback decision point*. Before such a step the agent may write its note, one line of prediction, and then acts:

```
<prediction>The mug is most likely in
a toaster, then a garbagecan, then a
cabinet.</prediction>
<action>go to toaster 1</action>
```

The environment's reply is appended as `<observation>...</observation>`. The whole episode forms one assistant stream after a fixed system prompt and the opening observation.

Why a counterfactual world. In standard ALFWorld, much of what an agent learns about object locations is shared across object classes (open surfaces first), and its value is modest. Our question needs knowledge that comes only from fine-tuning and changes which receptacle is worth visiting. ALF-CF enforces both.

Construction. ALF-CF assigns each learnable pair (scene type, target class) a convention receptacle class r_1 and a fallback r_2. The assignment uses the ALFWorld training games and the zero-shot location rankings of Qwen3-4B and Qwen3-8B. r_1 may not be one of the class's three most common receptacle types in the original data, must rank low for both models, and must rarely be the task's destination. No receptacle class may serve as the convention for more than 30% of a scene type's games, which stops an object-agnostic heuristic such as "check the garbage can first" from recovering the convention.

In each game, with probability 0.8, every instance of the target class is moved into one random instance of r_1, or of r_2 when r_1 is absent or is the destination. With probability 0.2 the original placement is kept.

Twelve of the 49 target classes are *unlearnable*: with probability 0.8 they are moved into a receptacle class drawn afresh in every game. They serve as a control in which the weights cannot hold a convention. Generation edits only the initial state of each game's PDDL problem, and both scripted teachers solve every generated game.

Splits. Floor plans are split per scene type into 21 for training, 3 for validation and 3 for test. ALFWorld's `valid_seen` and `valid_unseen` games are added to test. Each base game yields independently drawn variants: three in training, one in validation and two in test. This gives 8,148, 413 and 1,396 games.

Because `valid_seen` rooms come from training floor plans, the primary test population is the 1,116 games from held-out floor plans and `valid_unseen`. The 280 `valid_seen` games are reported separately.

Decision value of the convention. On the test games of learnable classes, knowing the object-specific convention saves 2.1 receptacle visits [1.9, 2.3] compared with the best object-agnostic ranking. The two rankings disagree at 82% of fallback decision points. A uniform search takes 12.2 visits to find the target and a searcher that knows the convention takes 2.0. Searching in the order of the zero-shot rankings of Qwen3-4B and Qwen3-8B would take 18.3 and 18.7 visits (on the test games from ALFWorld's validation sets). For unlearnable classes the object-specific advantage is 0.3 visits [−0.1, 0.6].

Arm	Change relative to N100	S_{data}
N100	none; every note is followed	1.00
NQ50OP	half of the real notes are foils; actions unchanged	0.51
NQ25OP	a quarter of the real notes are foils	0.75
NQ50	as NQ50OP, whole foil note unsupervised	0.51
SA50V2	half of all search actions follow the standard prior	0.66
N100M	no loss on actions taken without a note	1.00

Table 1: Training arms. The primary arms N100 and NQ50OP have eight seeds each; the other arms have three. A seed fixes both the random draws used to build the data and the data order.

Teachers and notes in the data. Training trajectories come from two scripted searchers that use only observations and remember what they have seen. T^+ visits receptacle types in the order of the location prior estimated on the ALF-CF training games, so it knows the convention. T^- uses the prior of the same games before the move. Once the target is in view, both finish the task by script.

At each fallback decision point the training data contain a real note with the top three remaining types under the ALF-CF prior (63%), a placebo in the same template that lists the remaining types alphabetically (7%), or no note (30%), drawn once per decision point. Because notes are absent at almost a third of decision points, removing one at test time stays within the training distribution.

4 Training Arms That Differ in One Respect

All arms fine-tune Qwen3-4B (Yang et al., 2025) with thinking disabled. The hyperparameters are identical across arms (learning rate 10^{-5}, one epoch, global batch 64, maximum length 12,288 tokens). The loss covers the agent's own tokens, that is the note lines and the actions. Observations are never supervised.

Each arm changes one thing relative to the reference arm N100.[1] Table 1 summarises them. We write S_{data} for the share of real-form notes in the training data whose top type is the receptacle type the teacher visits next.

[1]Arm names follow the registered protocol. N100 has fully reliable notes, NQq turns q% of the real notes into foils, the suffix OP marks a supervised opener, M marks a masked action loss, and SA50 makes 50% of search actions follow the standard prior (V2 is the protocol's version label).

N100. Trajectories come from T^+. Every real note states the convention, and the next action visits its top type. Note and convention never disagree, so the data cannot tell a learner whether the action follows the note or follows what the weights know about the object.

Contradicted notes: NQ50OP, NQ25OP, NQ50. These arms keep N100's trajectories and actions token for token. At a fraction q of real-note decision points ($q = 0.5$ or 0.25), the note is replaced by a *foil*, which is the convention ranking of another learnable class of the same scene type, written in the same template. The action still visits the true convention.

In NQ50OP and NQ25OP the foil's receptacle names are excluded from the loss, so the model never learns to write a foil. The opener and template of the foil remain supervised. As a result, the share of decision points at which writing a note is supervised is exactly that of N100: 0.69–0.71 depending on the draw. NQ50 excludes the whole foil note from the loss, which lowers that share to 0.39. Comparing NQ50 with NQ50OP checks whether writing fewer notes changes the result.

Action-side unreliability: SA50V2. Notes are always correct, but at every fallback decision point a coin decides whether the teacher takes the convention-following step or the standard-prior step. This happens whether or not a note is written, so the presence of a note does not predict a deviation. In the data the deviation rate is 0.50 with and without a note, and a χ^2 test of independence does not reject at any position or draw.

Standard-prior steps lengthen the search. The arm is therefore subsampled to the same number of fallback decision points as N100, about 6,600 of 8,148 trajectories.

No action loss without a note: N100M. N100M is N100 with the action loss removed at decision points that carry no note (30%). The model is never trained to act without a note. Section 2 compares this mask with the action-loss masking of Liu et al. (2026).

5 Measuring Control by the Written Note

Decision-point interventions. Each model plays every test game greedily, for at most 50 steps. At each fallback decision point with at least two receptacle types left to visit, we keep the model's own history and place one of several note lines before a prefilled `<action>`. We then sample the next action four times at temperature 0.7, using the same random numbers across conditions. The outcome is the receptacle type of the next `go to`.

Condition	Note line placed before the action
intact	the model's own note
flip	the note the same model writes when the task names another learnable class o' of the same scene type, renamed back to the target; o' is drawn uniformly among classes whose predicted top type is unvisited and differs from the intact top
remove	no note line
presence	the template with every type slot replaced by the word "receptacle" (no location, nothing to copy)
random	the template with a random order of the remaining types, a form no arm sees in training
placebo	the alphabetical placebo of the training data (manipulation check only)

Table 2: Conditions at a fallback decision point. Each is sampled four times at temperature 0.7 with common random numbers.

Table 2 lists the conditions.

At later decision points we also remove every earlier note from the history and repeat the intact and flip conditions.

At the episode level, we run counterfactual rollouts in which every note slot is forced to the template prediction for o' (*cf-foil*) or for the true class (*cf-true*), while the model chooses its own actions.

Estimands. Write $P_c(t)$ for the probability that the next `go to` under condition c targets receptacle type t. The primary estimand is the flip effect at the first fallback decision point of a game,

$$S^{\text{first}}_{\text{flip}} = P_{\text{flip}}(\text{foil top}) - P_{\text{intact}}(\text{foil top}),$$

averaged over games of learnable classes, one decision point per game. The first decision point is the only one at which every arm has the same context: no search evidence and no earlier note.

Later decision points are reported by position and by whether the convention receptacle has already been visited. They are never pooled across positions when arms are compared, because an arm that searches longer has more of them.

The removal effect is $P_{\text{intact}}(\text{intact top}) - P_{\text{remove}}(\text{intact top})$. The presence effect is $P_{\text{remove}}(\text{intact top}) - P_{\text{presence}}(\text{intact top})$, and we also report the flip effect measured against the presence condition. Resistance is $S^{\text{first}}_{\text{flip}}$ for unlearnable classes minus $S^{\text{first}}_{\text{flip}}$ for learnable classes. Because

the weights hold no convention for unlearnable classes, a positive resistance means the learned convention pushes back against a contradicting note. Every model's note-writing rate in free play is reported as a covariate.

A perfect-imitator reference. Some values of $S_{\text{flip}}^{\text{first}}$ follow from how the training data were built. We therefore compare each arm with S_{flip}^{*}, the flip effect of a policy that reproduces its training data's $P(\text{action type} \mid \text{observation}, \text{note})$ exactly.

In the three note-side arms, a foil note in training is always followed by the true convention. An imitator therefore ignores a flipped note, and $S_{\text{flip}}^{*} = 0$ (computed from the data in every draw).

In N100 and N100M, a flipped note never occurs in training and notes always agree with the convention. S_{flip}^{*} is undefined there. Its bounds are 0 if a tie goes to the convention and 1 if it goes to the note.

SA50V2 also never shows a foil. An imitator that credits its convention-following half to the note would follow a flipped note at the realised rate of convention-following steps, so $S_{\text{flip}}^{*} \in [0, 0.50]$.

The *copy bias* $S_{\text{flip}}^{\text{first}} - S_{\text{flip}}^{*}$ is a secondary, descriptive quantity. It is an interval for the arms whose reference is bounded.

Statistics and status. The unit of inference is the seed. Each seed is a separate construction draw and data order, with eight seeds for each primary arm and three for each other arm.

The primary contrast is N100 against NQ50OP on $S_{\text{flip}}^{\text{first}}$ in the primary test population (Welch t on seed values, two-sided, $\alpha = 0.05$). Eight secondary contrasts are Holm-corrected. Within a seed, intervals come from a bootstrap over games.

The protocol, the analysis script and the reading rules for NQ50OP were committed by hash to a private, timestamped repository before the test split was opened, and the hashed texts are released with the code. We call the primary contrast and the eight registered secondary tests confirmatory. The other read-outs are registered descriptive quantities.

6 Results

All numbers below are seed means on the primary test population (826 games of learnable classes and 290 of unlearnable classes), with 95% t intervals over seeds unless stated otherwise. Table 3 gives every arm, Table 4 the registered secondary contrasts and Figure 2 the first-decision flip effect by arm. The 280 `valid_seen` games give similar flip, removal and random-order effects in every arm (Appendix E); the one difference is noted under resistance. Appendix C lists every registered prediction with its outcome.

With always-followed data, the prediction steers the first decision without being needed. In N100 the agents write a prediction at 99.7% of fallback decision points, and its top type is the receptacle that holds the target in 76% of first decision points. Deleting the prediction changes almost nothing. The probability of moving to its top type drops by 0.015 [0.009, 0.022], and the agent still moves to the convention receptacle 96% of the time. A perfect imitator would not change its move either, because 30% of N100's training decision points teach the action without a note. What the data leave open is the flipped case.

Replacing the prediction with the one the same agent writes for another object moves the first step to that object's receptacle with $S_{\text{flip}}^{\text{first}} = 0.802$ [0.745, 0.858]. Across the eight seeds the effect ranges from 0.70 to 0.88. A content-free prediction ("a receptacle") leaves behaviour unchanged (presence effect −0.000), so the flip works through what the prediction says. The alphabetical placebo, a form after which the training data take the convention step rather than its top type, is ignored (effect 0.000).

The prediction need not be plausible. A randomly ordered ranking, a form never seen in training, is followed even more readily than the agent's own prediction for another object, with an effect of 0.904 [0.834, 0.974]. The random-order effect exceeds the flip effect in all eight seeds, by 0.103 [0.074, 0.132] on average. The flip effect is slightly smaller, not larger, for foils that rank high under the true object's convention (difference between the most and least plausible terciles: −0.069 [−0.104, −0.034]).

Knowing the convention does not reduce the effect. For unlearnable classes, whose location the weights cannot know, the flip effect is 0.810, so resistance (unlearnable minus learnable) is 0.008 [−0.034, 0.050] (H6). We had predicted a resistance of +0.03 to +0.12, and that prediction failed. On the `valid_seen` games, whose rooms come from training floor plans, unlearnable classes are redirected more often than learnable ones (0.883 against 0.817; difference 0.066 [0.023, 0.109]). We therefore claim no resistance only for held-out

Arm	Flip effect $S^{\text{first}}_{\text{flip}}$ learnable	unlearnable	Removal effect	Random order	Writes notes	Imitator S^{*}_{flip}	Copy bias
N100	0.802 [0.745, 0.858]	0.810	0.015	0.904	0.997	[0, 1]	[−0.20, 0.80]
NQ50OP	0.003 [0.002, 0.005]	0.017	0.004	0.008	0.995	0	0.003
NQ25OP	0.012 [0.005, 0.019]	0.028	0.004	0.017	0.995	0	0.012
NQ50	0.003 [−0.002, 0.008]	0.017	0.004	0.008	0.261	0	0.003
SA50V2	0.269 [0.153, 0.385]	0.192	0.057	0.297	0.998	[0, 0.50]	[−0.23, 0.27]
N100M	0.859 [0.780, 0.938]	0.830	0.211	0.971	0.999	[0, 1]	[−0.14, 0.86]

Table 3: Results by arm at the first fallback decision point (primary test population; seed means). The removal effect is the drop in the probability of the intact prediction's top type when the prediction line is deleted. The random-order column is the flip effect of a randomly ordered ranking, a form never seen in training. "Writes notes" is the share of fallback decision points at which the agent writes a prediction in free play. The copy bias is $S^{\text{first}}_{\text{flip}} - S^{*}_{\text{flip}}$, an interval where the imitator reference is only bounded.

rooms.

A flipped prediction never occurs in N100's training data, so a perfect imitator could either ignore it or follow it. The fine-tuned agents follow it, an inductive choice in a situation the data leave open.

The steering extends over whole episodes and is weaker at later steps. When the template prediction for another object is written at every prediction slot of an episode, the N100 agents go to that object's receptacle first in 79% of games. They visit 9.4 receptacles before seeing the target, against 2.1 when the true object's prediction is written. NQ50OP agents go to the foil's receptacle first in 1.8% of games (H7: difference 0.771 [0.717, 0.826]).

Later in the episode the flip effect is much smaller, but it does not fall steadily: 0.10 at the second decision point, 0.04 at the third and 0.27 at the fourth and later (learnable classes). It depends on what the search has shown. It is 0.34 while the convention receptacle is unvisited, 0.00 while some of its instances remain unsearched, and 0.18 once all of them have been searched without success. At these later points the context also holds failed visits and the agent's own earlier predictions. Removing the earlier predictions from the history raises the flip effect at those points from 0.125 to 0.216, a change of +0.091 [0.069, 0.113] (H8).

With contradicted predictions in the data, the agents match a perfect imitator (primary contrast). In NQ50OP, half of the real training predictions are replaced by another object's prediction while the actions stay those of N100. The flip effect is 0.003 [0.002, 0.005], against 0.802 for N100. The difference is 0.798 (Welch 95% CI [0.742, 0.854], $t = 33.6$, $p = 5 \times 10^{-9}$), the

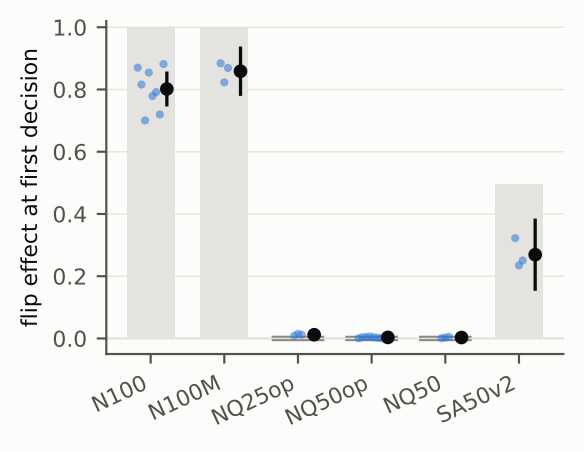


Figure 2: Flip effect at the first fallback decision point for learnable classes, by arm. Dots are seeds and black marks are arm means with 95% t intervals. Grey shows each arm's perfect-imitator reference S^{*}_{flip}: a line where it is defined (0 for the note-side arms), and a band where only its bounds are known.

registered primary result (predicted: at least 0.40). Because the imitator reference of NQ50OP is 0 (Section 5), the size of this contrast is set by N100.

The agents still write predictions at 99.5% of fallback decision points, and their predictions still name the convention (top-1 hit 0.73 at the first decision point). Under the reading rule fixed before the test, a flip effect below 0.15 with notes still written is read as reliability: the contradicted predictions, not a loss of note writing, account for the drop. This is the outcome the data construction implies.

The same holds for a quarter of contradicted predictions. NQ25OP reaches 0.012 [0.005, 0.019] (H5: −0.790 relative to N100). We had predicted 0.10 to 0.60. That prediction failed. NQ50, which also stops supervising the opening of foil predictions, writes notes at only 0.26 of decision points (we had predicted at most 0.10; the three seeds give

Contrast	Estimate	p	Holm
H7 episode, N100 − NQ50OP	0.771	4×10^{-9}	reject
H5 NQ25OP − N100	−0.790	5×10^{-9}	reject
H2 SA50V2 − N100	−0.532	1×10^{-5}	reject
H8 earlier notes off, N100	0.091	3×10^{-5}	reject
H3 N100M − N100	0.057	0.094	–
H4 P(conv. | removed), N100M − N100	−0.198	0.117	–
H6 resistance, N100	0.008	0.671	–
H1 NQ50OP − NQ50	0.000	0.822	–

Table 4: Registered secondary contrasts, Holm-corrected over the eight tests (family $\alpha = 0.05$). Estimates are differences in the first-decision flip effect except for H7 (first visit to the foil's receptacle in counterfactual rollouts), H8 (later flip effect, earlier predictions removed minus kept) and H4. Welch t on seed values; one-sample t for H6 and H8.

0.005, 0.36 and 0.42) and reaches 0.003, not distinguishable from NQ50OP (H1: difference 0.0003 [−0.004, 0.005]).

These arms sit within 0.012 of a perfect imitator of their data. In their training data a contradicting prediction is never followed by its top type, so the imitator reference is 0. Their flip effects are therefore set by the data. What they add about the model is that little copying tendency survives: the copy bias is 0.003 [0.002, 0.005] in NQ50OP and 0.012 [0.005, 0.019] in NQ25OP, above zero but small.

Unreliable actions leave part of the steering. In SA50V2 the predictions are always correct, but half of all search steps follow the standard prior whether or not a prediction is written. The flip effect is 0.269 [0.153, 0.385] (H2: −0.532 relative to N100). These agents follow their own intact prediction at the first step only 64% of the time, and relative to that rate the flip effect is 0.42. The content-free prediction changes little (presence effect 0.012), which confirms that in these data the presence of a prediction does not signal a deviation.

A flipped prediction never occurs in this arm's training data either. Its imitator reference therefore lies between 0, for an imitator that credits the correct steps to the weights, and 0.50, for one that credits them to the prediction. The flip effect lies below the reference of the note-crediting imitator (copy bias −0.227 [−0.334, −0.121]) and above that of the weight-crediting one (+0.269 [0.153, 0.385]), so the agents match neither. The registered comparison with NQ50OP's copy bias, outside the Holm family, gives 0.231 [0.124, 0.337] ($p = 0.011$) against the upper reference and −0.266 [−0.382, −0.150] ($p = 0.010$) against the lower one.

Withholding the action loss on note-free steps does not detectably change the steering. N100M receives no action loss at decision points without a prediction. Its flip effect is 0.859 [0.780, 0.938], not distinguishable from N100 (H3: +0.057, $p = 0.094$).

Deleting the prediction now matters more. The removal effect is 0.211, and the agent goes to the convention receptacle without a prediction in 76% of first steps, against 96% for N100. With three seeds this difference is not significant (H4: −0.198, $p = 0.117$). The content-free prediction raises the convention rate relative to no prediction (presence effect −0.178), consistent with an agent that was never trained to act without a note.

7 Conclusion

We asked whether an agent's written prediction controls its next action when the knowledge it states has been learned into the weights, and what in the training data decides this. In ALF-CF, where object locations can only be learned from fine-tuning, the answer depends on the data. Qwen3-4B agents fine-tuned on data in which every prediction is followed copy their prediction into the first search step, although deleting the prediction changes almost nothing. When a quarter or half of the training predictions are contradicted by the action, the agents keep writing predictions but follow a replaced one in at most about 1% of first steps, as an imitator of such data would.

Two implications follow for work that reads or edits agents' written reasoning. First, deletion and editing answer different questions. Because our data also teach acting without a prediction, a deletion test would have called the predictions of the always-followed agents decorative, while writing another object's prediction at every step steers them through the whole episode. Second, in our setting the training data decided whether a written prediction controlled the decision. Agents whose training predictions were always borne out by the next action followed a replaced prediction at about 80% of first steps and a randomly ordered one at 90%. Contradicting a quarter of the training predictions removed this, as the data imply, while the agents kept writing predictions.

Limitations

The study uses one model family and size (Qwen3-4B), supervised fine-tuning only, and a single counterfactual environment derived from ALFWorld. Reinforcement learning after fine-tuning could change how much the prediction is used, as Liu et al. (2026) report for reasoning traces.

The predictions are one-line templates that name a receptacle type, and the action names a receptacle of that type. Following such a prediction can be implemented as copying, and the high flip effect for randomly ordered predictions is consistent with that. Free-form or paraphrased predictions may behave differently.

The conventions are arbitrary by construction, and the teachers are scripted searchers. Both choices make the knowledge identifiable but less natural than the knowledge of a deployed agent.

The secondary arms have three seeds each, so the contrasts H3 and H4 have little power. The imitator reference is only bounded for N100, N100M and SA50V2, so their copy biases are intervals.

The first decision point was chosen as the primary estimand after a validation pilot and was then confirmed on the held-out test split.

References

Sanjeda Akter, Ibne Farabi Shihab, and Anuj Sharma. 2025. Causal consistency regularization: Training verifiably sensitive reasoning in large language models. *Preprint*, arXiv:2509.01544.

Marc-Alexandre Côté et al. 2018. TextWorld: A learning environment for text-based games. *Preprint*, arXiv:1806.11532.

Scott Emmons, Erik Jenner, David K. Elson, Rif A. Saurous, Senthooran Rajamanoharan, Heng Chen, Irhum Shafkat, and Rohin Shah. 2025. When chain of thought is necessary, language models struggle to evade monitors. *Preprint*, arXiv:2507.05246.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673.

Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Dohyung Kim, Sangmook Lee, Youngchul Sung, and Kyomin Jung. 2025. ReflAct: World-grounded decision making in LLM agents via goal-state reflection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Tomek Korbak et al. 2025. Chain of thought monitorability: A new and fragile opportunity for AI safety. *Preprint*, arXiv:2507.11473.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. 2023. Measuring faithfulness in chain-of-thought reasoning. *Preprint*, arXiv:2307.13702.

Jingyu Liu, Zhiwen Wang, Yuxin Jing, Huanyu Zhou, and Yong Liu. 2026. Where do CoT training gains land in LLM based agents? *Preprint*, arXiv:2606.26935.

Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063.

Charles Lovering, Rohan Jha, Tal Linzen, and Ellie Pavlick. 2021. Predicting inductive biases of pre-trained models. In *International Conference on Learning Representations*.

Debjit Paul, Robert West, Antoine Bosselut, and Boi Faltings. 2024. Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15012–15032.

Mohammadreza Sediqin and Zahra Fatemi. 2026. Causal intervention on agent reasoning: Not all reasoning is load-bearing. NeurIPS 2026 Workshop "Who Verifies the Agents? Toward Reliable Agent Development".

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

Anand Swaroop, Akshat Nallani, Saksham Uboweja, Adiliia Uzdenova, Michael Nguyen, Kevin Zhu, Sunishchal Dev, Ashwinee Panda, Vasu Sharma, and Maheep Chaudhary. 2025. FRIT: Using causal importance to improve chain-of-thought faithfulness. *Preprint*, arXiv:2509.13334v1.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2023. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Hanlin Wang, Chak Tou Leong, Jian Wang, and Wenjie Li. 2026. Seeing isn't believing: Mitigating belief inertia via active intervention in embodied agents. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Yu Wang. 2026. The dark room in the reward channel: Dense prediction rewards collapse GRPO-trained LLM agents – and the channel, not the content, decides what works. *Preprint*, arXiv:2607.21273.

Alex Warstadt, Yian Zhang, Xiaocheng Li, Haokun Liu, and Samuel R. Bowman. 2020. Learning which features matter: RoBERTa acquires a preference for linguistic generalizations (eventually). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 217–235.

Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. 2024. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*.

Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *The Twelfth International Conference on Learning Representations*.

An Yang et al. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Kai Zhang, Xiangchao Chen, Bo Liu, et al. 2025. Agent learning via early experience. *Preprint*, arXiv:2510.08558.

A ALF-CF Construction Details

Source games. We use the ALFWorld `json_2.1.1` games of the six standard task types, without the `movable` and `Sliced` variants: 3,553 training, 140 `valid_seen` and 134 `valid_unseen` games. Scene types follow the floor-plan numbering (kitchen, living room, bedroom, bathroom).

Convention rules. For every learnable pair (scene type, class), r_1 is chosen greedily, starting with the classes that have the most training games. A candidate must appear in enough of the scene type's floor plans, must not be among the class's three most common receptacle types, and must have a destination share below 0.10. Among the candidates, the choice minimises a cost that penalises the receptacle type's current usage, its normalised rank in the data prior, its rank in the zero-shot rankings of the two Qwen3 models, and its number of instances.

When no candidate passes the strictest thresholds, a recorded relaxation ladder loosens the presence, data-rank and model-rank thresholds step by step. All kitchen and bathroom classes are assigned at the strictest level, while living-room and bedroom classes need up to the third and the fifth of five levels.

r_2 is the cheapest other class at the same level that appears in at least 90% of floor plans. The rules, the per-class candidate tables and the chosen conventions are released as `alfcf/rules_v1.json`.

Unlearnable classes. Twelve of the 49 target classes (20% of training games) were drawn at random, subject to every scene type having one with at least 20 training games: apple, butterknife, cd, laptop, peppershaker, remotecontrol, soapbottle, spoon, spraybottle, tissuebox, vase and wateringcan.

B Training Data

The note template is `The O is most likely in a T1, then a T2, then a T3.`, with fewer slots when fewer receptacle types remain. The teachers' location priors are type-level frequencies estimated on the ALF-CF training games. Ties are broken by the scene type's object-agnostic frequency and then by name.

Data construction was checked per arm and draw before training. The checks covered identical action sequences across the note-side arms, equal supervised note-writing shares, mask positions, foil rates, deviation rates by note presence, and decision-point matching. The reports are released with the code.

C Registered Predictions and Outcomes

Table 5 lists every prediction registered for the main study, with its outcome on the primary test popula-

Quantity	Prediction	Result	Outcome
Primary: N100 − NQ50OP, $S^{\text{first}}_{\text{flip}}$	≥ 0.40, $p < 0.05$	0.798, $p = 5 \times 10^{-9}$	holds
Reading of NQ50OP	below 0.15 most likely	0.003	reliability
H1 NQ50OP − NQ50	–	0.000, $p = 0.82$	–
H2 SA50V2, $S^{\text{first}}_{\text{flip}}$	0.15 to 0.45	0.269	holds
H3 N100M, $S^{\text{first}}_{\text{flip}}$	0.80 to 0.95	0.859	holds
H4 P(conv. | removed), N100M − N100	at most −0.05	−0.198, $p = 0.12$	in range; not significant
H5 NQ25OP, $S^{\text{first}}_{\text{flip}}$	0.10 to 0.60	0.012	fails
H6 resistance, N100	+0.03 to +0.12	0.008 [−0.034, 0.050]	fails
H7 episode, N100 − NQ50OP	–	0.771, $p = 4 \times 10^{-9}$	–
H8 earlier notes off, N100	+0.05 to +0.15	0.091	holds
NQ50, $S^{\text{first}}_{\text{flip}}$	−0.05 to 0.05	0.003	holds
SA50V2, presence effect	$\lvert\cdot\rvert < 0.05$	0.012	holds
SA50V2, P(intact top | intact)	0.50 to 0.75	0.637	holds
N100, $S^{\text{first}}_{\text{flip}}$ mean	0.75 to 0.88	0.802	holds
N100, seed SD of $S^{\text{first}}_{\text{flip}}$	≤ 0.08	0.067	holds
N100, random-order effect	≥ 0.85	0.904	holds
N100, plausibility slope	$\lvert\cdot\rvert < 0.10$	−0.069	holds
N100, removal effect	$\lvert\cdot\rvert < 0.05$	0.015	holds
N100, visits, cf-foil − cf-true	$\geq +5$	+7.3	holds
Note writing, N100	≥ 0.95	0.997	holds
Note writing, NQ50OP	≥ 0.90	0.995	holds
Note writing, NQ50	≤ 0.10	0.261 (seeds 0.005, 0.36, 0.42)	fails
Copy bias, NQ50OP − SA50V2, upper ref.	–	0.231, $p = 0.011$	–
Copy bias, NQ50OP − SA50V2, lower ref.	–	−0.266, $p = 0.010$	–

Table 5: Registered predictions and outcomes. H1–H8 are Holm-corrected as a family (Table 4); the copy-bias comparison lies outside that family. For H4 the registered prediction was a drop of at least 0.05 (stated with probability 0.6); the estimate is in that range, but the test is not significant.

tion. Rows without a prediction are registered tests whose direction was not predicted.

D Evaluation Details

The system prompt describes the action format and the optional prediction line. Rollouts use greedy decoding with a stop string after each action, and the environment's reply is appended as an observation.

The model's own note for another class o' is generated by renaming the target to o' in the task line and in the model's earlier notes, prefilling `<prediction>The` o' `is most likely in a`, decoding greedily, and renaming the result back to the target. We checked that each prefill boundary tokenises in the same way as in training.

Counterfactual rollouts insert the forced note whenever the target has not been seen and the previous observation neither reports a closed receptacle nor reads "Nothing happens". These are exactly the slots where training data contain notes.

E Additional Results

Table 6 repeats the main read-outs on the 280 `valid_seen` test games, whose rooms come from training floor plans. The flip, removal and random-order effects are close to those on the primary population in every arm. The exception is the learnable–unlearnable difference in N100, which is 0.066

Arm	$S^{\text{first}}_{\text{flip}}$ learn.	$S^{\text{first}}_{\text{flip}}$ unlearn.	Removal effect	Random order	Writes notes
N100	0.817	0.883	0.011	0.881	0.997
NQ50OP	0.004	0.005	0.002	0.002	0.998
NQ25OP	0.013	0.014	0.005	0.008	0.999
NQ50	0.004	0.006	0.002	0.000	0.314
SA50V2	0.246	0.163	0.065	0.290	0.991
N100M	0.869	0.879	0.247	0.966	0.999

Table 6: Read-outs on the `valid_seen` test games (seed means).

Arm	Decision point 2nd	3rd	4th+	Convention receptacle unvis.	partial	searched
N100	0.10	0.04	0.27	0.34	0.00	0.18
NQ50OP	0.02	0.02	0.04	0.07	0.00	0.03
NQ25OP	0.03	0.01	0.06	0.22	0.00	0.04
NQ50	0.02	0.02	0.08	0.11	0.00	0.06
SA50V2	0.09	0.04	0.08	0.08	0.02	0.13
N100M	0.15	0.06	0.17	0.44	0.00	0.19

Table 7: Flip effect at later decision points for learnable classes (primary test population; seed means of the per-model read-outs). The right block splits the same decision points by whether the convention receptacle is unvisited, partly searched, or fully searched without success. The right block pools positions and is descriptive.

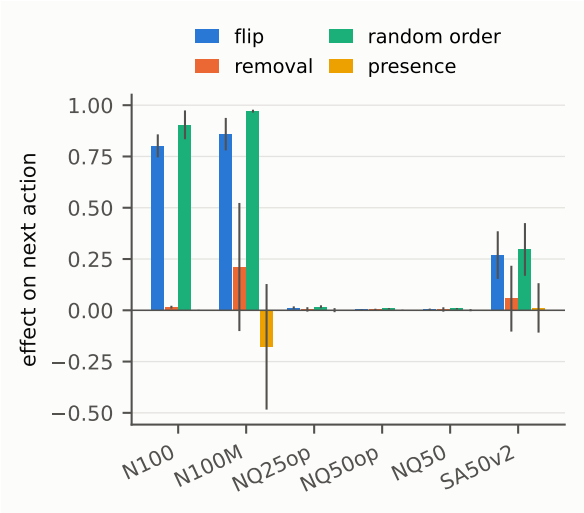


Figure 3: Flip, removal, random-order and presence effects at the first decision point by arm (seed means, 95% t intervals; primary test population).

here and 0.008 on the primary population.

At the first decision point, the flip based on the model's own prediction and the flip based on the template ranking for the same foil class differ by -0.055 in N100 (the template is followed slightly more often), -0.051 in N100M, -0.015 in SA50V2, and 0.000 in NQ50OP (this checks an assumption of the imitator reference).

Table 7 gives the flip effect at later decision points for every arm, and Figure 3 shows the flip, removal, random-order and presence effects together.